

Kernel Density Estimation — Effetto della Bandwidth

2026-04-22

Contents

1	Introduzione	1
2	Setup: la distribuzione vera e il campione	2
2.1	La mistura bimodale	2
2.2	Visualizzazione del campione e della vera densità	2
3	Effetto della bandwidth: panoramica	3
3.1	La regola di Silverman come riferimento	3
3.2	Confronto tra cinque valori di h	3
4	Il tradeoff bias-varianza	5
4.1	Tre casi esemplari a confronto	5
5	La curva ISE: trovare h ottimale	8
5.1	Calcolo dell'ISE al variare di h	8
5.2	Visualizzazione della curva a U	8
6	Conclusioni	9

```
library(ggplot2)
library(patchwork)
```

1 Introduzione

La **Kernel Density Estimation (KDE)** è un metodo non parametrico per stimare la densità di probabilità di una variabile aleatoria a partire da un campione osservato, senza assumere nessuna forma funzionale (es. normalità).

Dato un campione $x_1, \dots, x_n$, la KDE è definita come:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

dove:

- $K(\cdot)$ è il **kernel**, una funzione simmetrica e non negativa con integrale 1 (in questo notebook usiamo il kernel gaussiano)
- $h > 0$ è la **bandwidth**, il parametro che controlla quanto è “liscia” la stima

L'obiettivo di questo notebook è mostrare empiricamente come la scelta di h influenzi la qualità della stima, usando una distribuzione volutamente “difficile”: una **mistura bimodale**.

2 Setup: la distribuzione vera e il campione

2.1 La mistura bimodale

Usiamo come distribuzione generatrice una mistura di due gaussiane:

$$f(x) = 0.4 \cdot \mathcal{N}(-2, 1) + 0.6 \cdot \mathcal{N}(3, 0.5)$$

Questa distribuzione è un caso difficile per la KDE perché ha **due picchi ben separati** e **varianze diverse**. Una bandwidth unica non può adattarsi perfettamente a entrambi.

```
set.seed(42)

# Vera densità
true_density <- function(x) {
  0.4 * dnorm(x, mean = -2, sd = 1) +
  0.6 * dnorm(x, mean = 3, sd = 0.5)
}

# Campionamento dalla mistura
sample_mixture <- function(n) {
  comp <- sample(c(1, 2), size = n, replace = TRUE, prob = c(0.4, 0.6))
  ifelse(comp == 1, rnorm(n, -2, 1), rnorm(n, 3, 0.5))
}

# KDE con kernel gaussiano - implementazione manuale
kde_manual <- function(x_grid, data, h) {
  sapply(x_grid, function(x) mean(dnorm((x - data) / h)) / h)
}

# Parametri globali
n <- 300
data <- sample_mixture(n)
x_grid <- seq(-6, 6.5, length.out = 600)
```

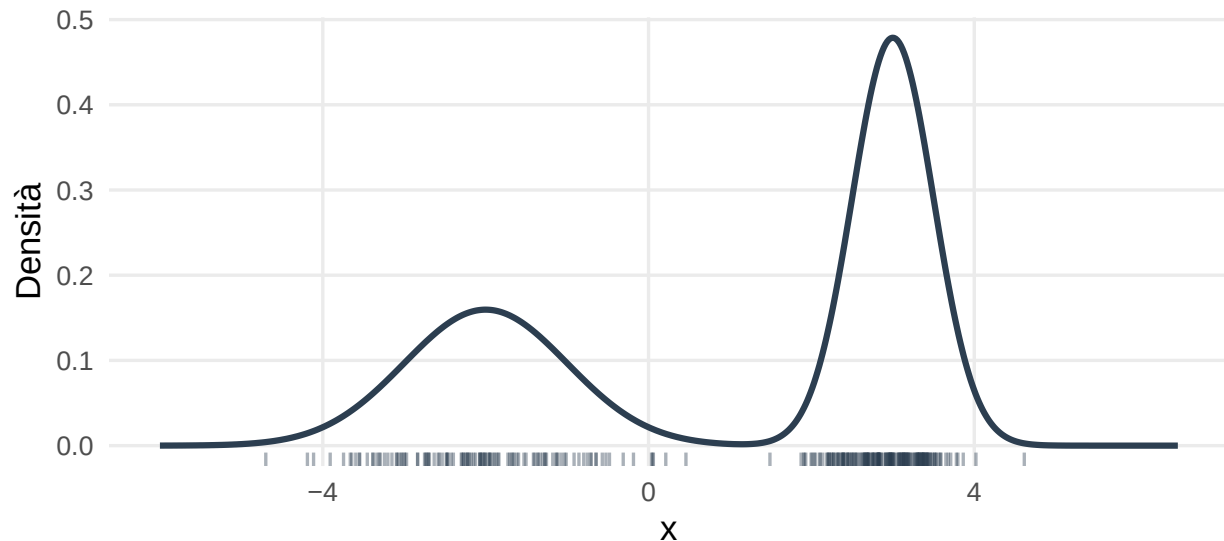
2.2 Visualizzazione del campione e della vera densità

```
df_true <- data.frame(x = x_grid, y = true_density(x_grid))

ggplot() +
  geom_rug(aes(x = data), alpha = 0.4, color = "#2c3e50") +
  geom_line(data = df_true, aes(x = x, y = y),
            color = "#2c3e50", linewidth = 1.1) +
  labs(
    title = "Vera densità e campione osservato",
    subtitle = paste0("f(x) = 0.4·N(-2,1) + 0.6·N(3,0.5) | n = ", n),
    x = "x", y = "Densità"
  ) +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"),
        panel.grid.minor = element_blank())
```

Vera densità e campione osservato

$$f(x) = 0.4 \cdot N(-2, 1) + 0.6 \cdot N(3, 0.5) \mid n = 300$$



Il `geom_rug` sul fondo mostra la posizione delle singole osservazioni. Si vede chiaramente la struttura bimodale: un addensamento intorno a -2 e uno più stretto intorno a 3 .

3 Effetto della bandwidth: panoramica

3.1 La regola di Silverman come riferimento

Prima di variare h manualmente, calcoliamo la bandwidth suggerita dalla **regola di Silverman**, che è il default di R (`bw.nrd0`):

$$h_{\text{Silverman}} = 1.06 \cdot \hat{\sigma} \cdot n^{-1/5}$$

Questa formula è ottimale se i dati sono gaussiani. Su distribuzioni multimodali tende a **sovrastimare** h perché $\hat{\sigma}$ è gonfiata dalla distanza tra i due picchi.

```
h_silverman <- bw.nrd0(data)
cat("h di Silverman:", round(h_silverman, 4))
```

```
## h di Silverman: 0.7324
```

3.2 Confronto tra cinque valori di h

Confrontiamo la KDE con cinque valori di h diversi, più la vera densità in tratteggio.

```
h_values <- c(0.10, 0.20, h_silverman, 0.80, 1.80)
h_labels <- c(
  "h = 0.10 (troppo piccolo)",
  "h = 0.20 (un po' piccolo)",
  paste0("h = ", round(h_silverman, 2), " (Silverman)"),
  "h = 0.80 (un po' grande)",
  "h = 1.80 (troppo grande)"
```

```

)
colori <- c("#c0392b", "#e67e22", "#27ae60", "#2980b9", "#8e44ad")

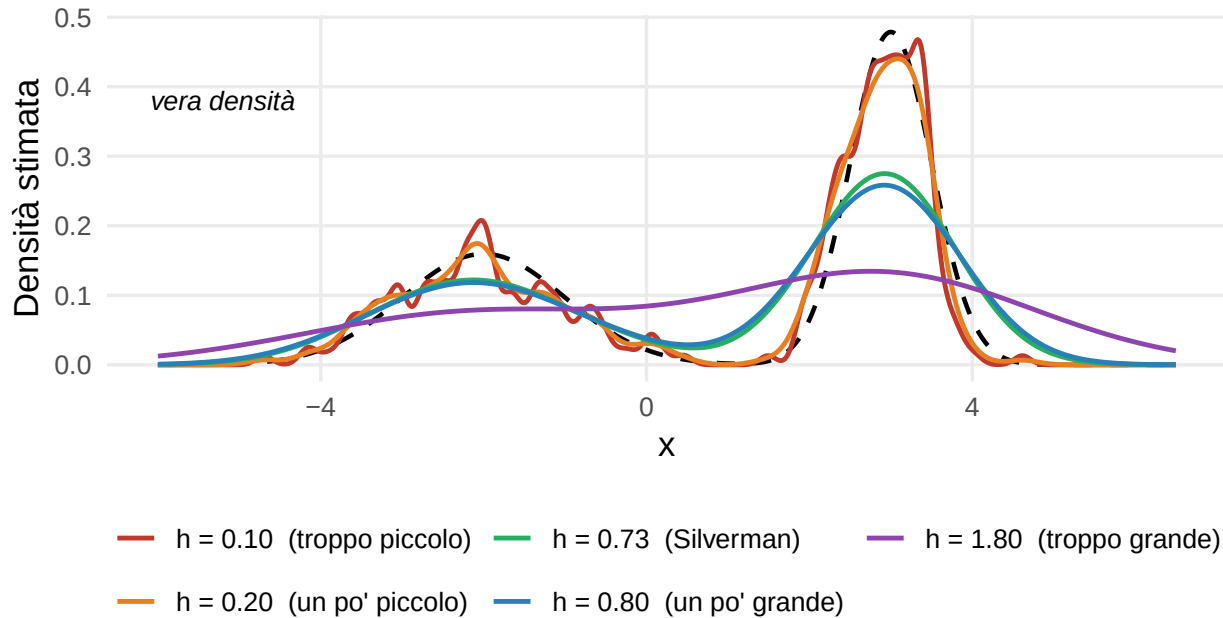
df_all <- do.call(rbind, lapply(seq_along(h_values), function(i) {
  data.frame(
    x      = x_grid,
    y      = kde_manual(x_grid, data, h_values[i]),
    label = h_labels[i]
  )
}))
df_all$label <- factor(df_all$label, levels = h_labels)

ggplot() +
  geom_line(data = df_true, aes(x = x, y = y),
            color = "black", linetype = "dashed", linewidth = 0.8) +
  geom_line(data = df_all, aes(x = x, y = y, color = label),
            linewidth = 0.85) +
  scale_color_manual(values = setNames(colori, h_labels)) +
  annotate("text", x = -5.2, y = 0.38, label = "vera densità",
           size = 3.5, color = "black", fontface = "italic") +
  labs(
    title      = "KDE al variare di h",
    subtitle   = paste0("f(x) = 0.4·N(-2,1) + 0.6·N(3,0.5) | n = ", n),
    x = "x", y = "Densità stimata", color = NULL
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom",
        legend.text     = element_text(size = 10),
        plot.title       = element_text(face = "bold"),
        panel.grid.minor = element_blank()) +
  guides(color = guide_legend(nrow = 2))

```

KDE al variare di h

$$f(x) = 0.4 \cdot N(-2, 1) + 0.6 \cdot N(3, 0.5) \mid n = 300$$



Si osserva chiaramente:

- Con h piccolo (rosso) la stima è **irregolare**: ogni osservazione forma un piccolo picco, il rumore campionario domina (*overfitting*)
- Con h grande (viola) la stima è **troppo liscia**: i due picchi si fondono in uno solo, la struttura bimodale scompare (*underfitting*)
- La regola di Silverman (verde) bilancia i due estremi, ma tende già a smussare un po' troppo il secondo picco (quello più stretto a destra)

4 Il tradeoff bias-varianza

4.1 Tre casi esemplari a confronto

Il grafico seguente mostra i tre casi paradigmatici affiancati, con l'area sotto la curva colorata per rendere più immediato il confronto visivo.

```
make_panel <- function(h, titolo, sottotitolo, col) {  
  df <- data.frame(x = x_grid, y = kde_manual(x_grid, data, h))  
  ggplot() +  
    geom_ribbon(data = df, aes(x = x, ymin = 0, ymax = y),  
              fill = col, alpha = 0.15) +  
    geom_line(data = df, aes(x = x, y = y),  
             color = col, linewidth = 1.0) +  
    geom_line(data = df_true, aes(x = x, y = true_density(x)),  
             color = "black", linetype = "dashed", linewidth = 0.7) +  
    labs(title = titolo, subtitle = sottotitolo, x = "x", y = "Densità") +  
    ylim(0, 0.55) +  
    theme_minimal(base_size = 11) +
```

```

    theme(plot.title      = element_text(face = "bold", color = col, size = 12),
          plot.subtitle   = element_text(size = 9, color = "gray30"),
          panel.grid.minor = element_blank())
  }

p1 <- make_panel(0.10, "Undersmoothing", "h = 0.10 -> overfitting", "#c0392b")
p2 <- make_panel(h_silverman,
                 "Silverman",
                 paste0("h = ", round(h_silverman, 2), " -> buon compromesso"),
                 "#27ae60")
p3 <- make_panel(1.80, "Oversmoothing", "h = 1.80 -> underfitting", "#8e44ad")

# Layout verticale (ncol=1): ogni pannello occupa tutta la larghezza
(p1 / p2 / p3) +
  plot_annotation(
    title      = "Bias-varianza tradeoff nella KDE",
    subtitle   = paste0("Vera densità in tratteggio | n = ", n),
    theme      = theme(plot.title      = element_text(face = "bold", size = 13),
                      plot.subtitle   = element_text(size = 10, color = "gray30"))
  )

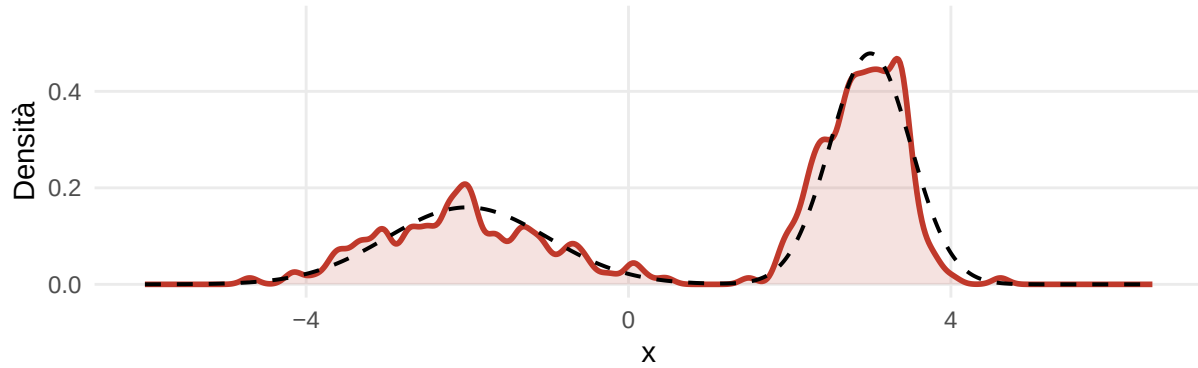
```

Bias-varianza tradeoff nella KDE

Vera densità in tratteggio | $n = 300$

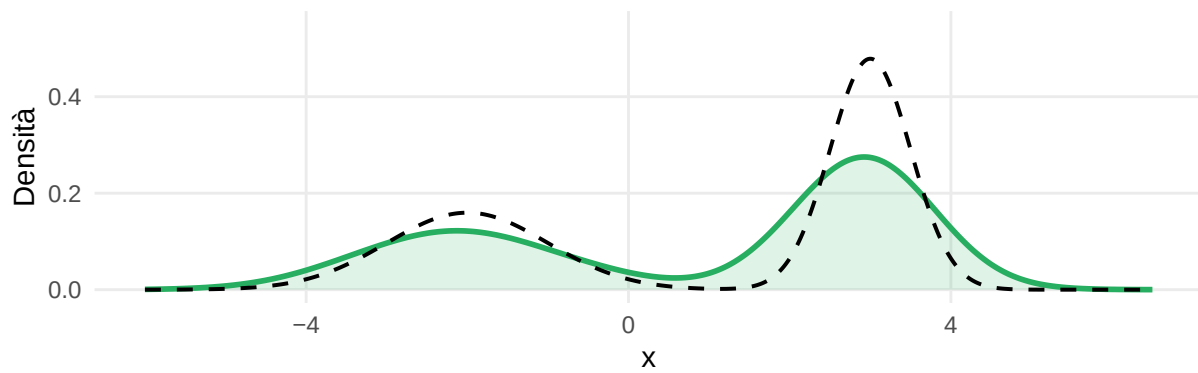
Undersmoothing

$h = 0.10 \rightarrow$ overfitting



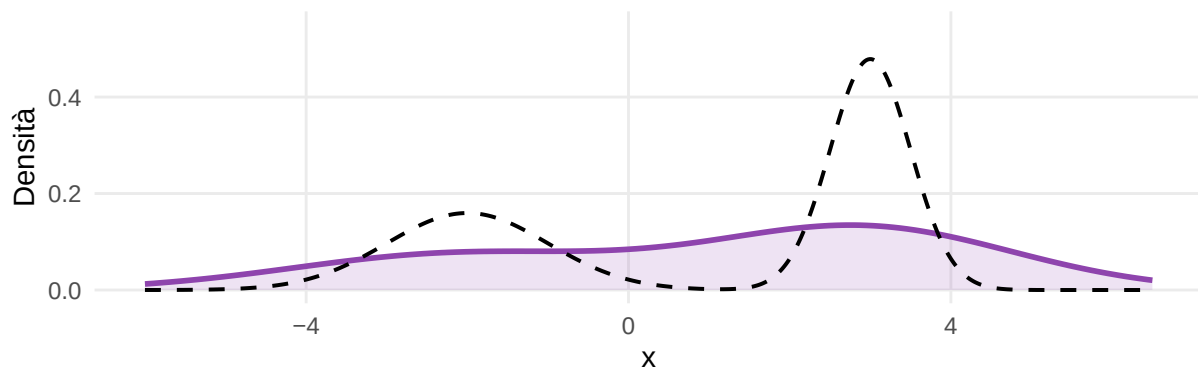
Silverman

$h = 0.73 \rightarrow$ buon compromesso



Oversmoothing

$h = 1.80 \rightarrow$ underfitting



Il tradeoff si può formalizzare scomponendo l'errore quadratico medio integrato (**MISE**):

$$\text{MISE}(\hat{f}) \approx \underbrace{\frac{h^4}{4} \left(\int u^2 K(u) du \right)^2 \int [f''(x)]^2 dx}_{\text{bias}^2 \uparrow \text{ con } h} + \underbrace{\frac{1}{nh} \int K^2(u) du}_{\text{varianza} \downarrow \text{ con } h}$$

- Il **bias** cresce con h : una finestra larga appiattisce le strutture fini
- La **varianza** decresce con h : una finestra larga aggrega più osservazioni, riducendo il rumore
- Il valore ottimale $h^* \propto n^{-1/5}$ bilancia i due termini

5 La curva ISE: trovare h ottimale

5.1 Calcolo dell'ISE al variare di h

Poiché conosciamo la vera densità (siamo in un contesto simulato), possiamo calcolare direttamente l'**Integrated Squared Error**:

$$\text{ISE}(h) = \int [\hat{f}(x) - f(x)]^2 dx \approx \sum_j [\hat{f}(x_j) - f(x_j)]^2 \Delta x$$

In un problema reale la vera f non è nota, e si ricorre alla cross-validation per stimare l'ISE. Qui lo usiamo per visualizzare il concetto.

```
h_seq <- seq(0.05, 2.5, length.out = 120)
dx <- diff(x_grid)[1]

ise_values <- sapply(h_seq, function(h) {
  f_hat <- kde_manual(x_grid, data, h)
  f_true <- true_density(x_grid)
  sum((f_hat - f_true)^2) * dx
})

h_best <- h_seq[which.min(ise_values)]
cat("h ottimale (min ISE):", round(h_best, 3), "\n")

## h ottimale (min ISE): 0.235

cat("h Silverman:", round(h_silverman, 3), "\n")

## h Silverman: 0.732

cat("Differenza:", round(abs(h_best - h_silverman), 3))

## Differenza: 0.497
```

5.2 Visualizzazione della curva a U

```
df_ise <- data.frame(h = h_seq, ISE = ise_values)

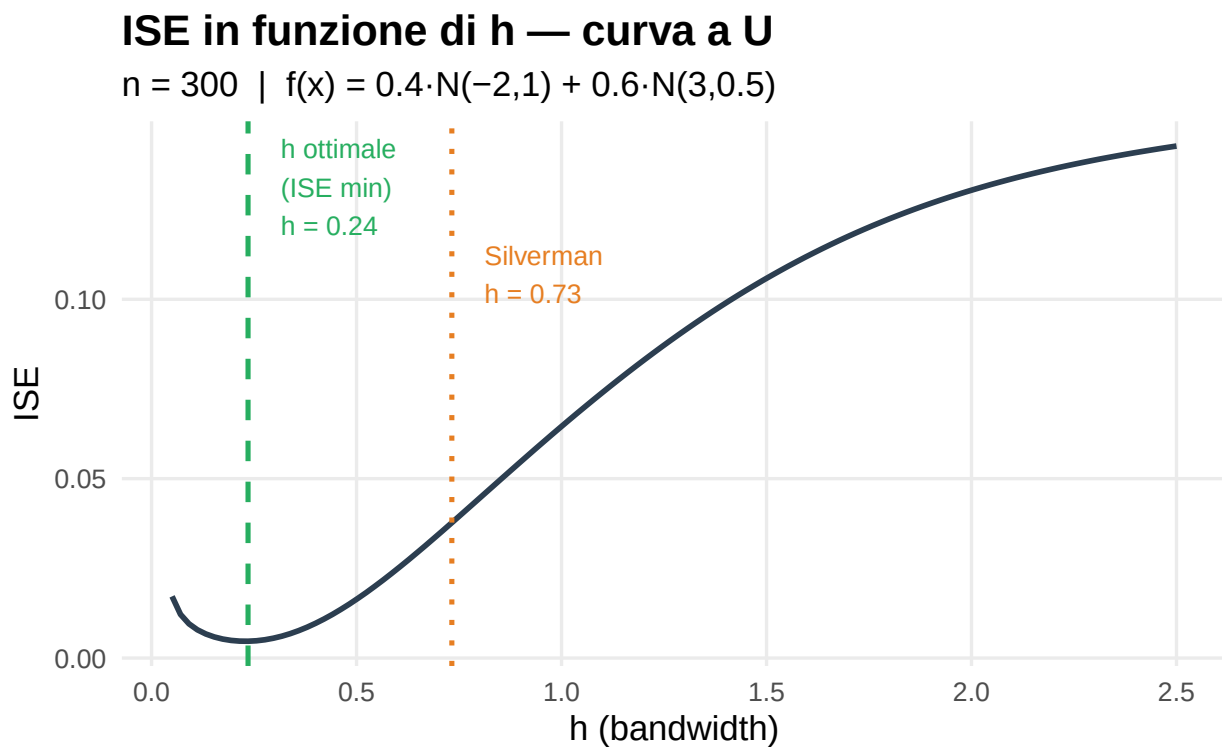
ggplot(df_ise, aes(x = h, y = ISE)) +
  geom_line(color = "#2c3e50", linewidth = 1.0) +
  geom_vline(xintercept = h_best, color = "#27ae60",
            linetype = "dashed", linewidth = 0.9) +
  geom_vline(xintercept = h_silverman, color = "#e67e22",
            linetype = "dotted", linewidth = 0.9) +
```



```

annotate("text", x = h_best + 0.08, y = max(ise_values) * 0.92,
        label = paste0("h ottimale\n(ISE min)\nh = ", round(h_best, 2)),
        color = "#27ae60", size = 3.5, hjust = 0) +
annotate("text", x = h_silverman + 0.08, y = max(ise_values) * 0.75,
        label = paste0("Silverman\nh = ", round(h_silverman, 2)),
        color = "#e67e22", size = 3.5, hjust = 0) +
labs(
  title = "ISE in funzione di h - curva a U",
  subtitle = paste0("n = ", n, " | f(x) = 0.4·N(-2,1) + 0.6·N(3,0.5)"),
  x = "h (bandwidth)", y = "ISE"
) +
theme_minimal(base_size = 13) +
theme(plot.title = element_text(face = "bold"),
      panel.grid.minor = element_blank())

```



La curva ISE ha la caratteristica **forma a U**:

- A sinistra del minimo: h troppo piccolo $\rightarrow$ varianza alta $\rightarrow$ ISE alto
- A destra del minimo: h troppo grande $\rightarrow$ bias alto $\rightarrow$ ISE alto
- Il minimo identifica il h ottimale per questo campione specifico

La regola di Silverman (linea arancione tratteggiata) si posiziona vicino al minimo ma leggermente a destra, confermando la sua tendenza all'oversmoothing su distribuzioni non gaussiane.

6 Conclusioni

	h piccolo	h ottimale	h grande
Bias	Basso	Bilanciato	Alto
Varianza	Alta	Bilanciata	Bassa
Aspetto	Rumoroso, irregolare	Segue la struttura vera	Troppo liscio, unimodale
ISE	Alto	Minimo	Alto

La scelta della bandwidth è il problema centrale della KDE. La regola di Silverman è un buon punto di partenza ma **non è ottimale su distribuzioni multimodali o asimmetriche**, proprio perché assume implicitamente la gaussianità dei dati attraverso $\hat{\sigma}$.

In pratica, metodi come **Sheather-Jones** (`bw.SJ` in R) o la **cross-validation** forniscono stime di h più robuste perché non fanno assunzioni sulla forma della distribuzione.